

Introduction to oneDPL

Dr. Pablo Reble



Learning Objectives

- Methods and concepts expressing Parallel Pattern in C++
- C++ STL Algorithms and Execution Policies
 - ISO C++ for device programming
 - Custom iterators to expand applicability
- Making use of libraries to improve productivity and performance 😊
 - Write code once, run anywhere.



oneAPI Industry Initiative

Freedom to Make Your Best Choice

- One programming model for multiple architectures and vendors
- Cross-architecture code reuse for freedom from vendor lock-in

Realize all the Hardware Value

- Performance across CPU, GPUs, FPGAs, and other accelerators
- Expose and exploit cutting-edge features of the latest hardware

Develop & Deploy Software with Peace of Mind

- Open industry standards provide a safe, clear path to the future
- Compatible with existing languages and programming models including C, C++ with SYCL*, Python, OpenMP, Fortran, and MPI
- Powerful libraries for acceleration of domain-specific functions



Intel® oneAPI DPC++ Library (oneDPL)

- High-productivity APIs for heterogeneous computing – CPUs, GPUs, FPGAs, and more
- APIs are based on standards and familiar extensions– C++ STL, Boost.Compute*, and SYCL
- Optimized C++ standard algorithms – parallel algorithms (C++17) and utilities
- SYCLomatic integration – complements DPC++ components to simplify migration of CUDA* applications using Thrust* API to oneDPL

oneDPL Specification & Reference Implementation



Spec: oneDPL section covers:

- Parallelized C++17 algorithms with SYCL* backend support
- Tested C++ standard APIs that work in SYCL* kernels
- Execution Policies
- Custom Utilities and Algorithms



<https://spec.oneapi.io/versions/latest/elements/oneDPL/source/index.html#onedpl-section>



Source
Code

oneDPL implementation

- Available as part of the Intel® oneAPI Base Toolkit
- Header-only library – relies on DPC++ and oneTBB runtime libraries
- Supports multiple backends (oneTBB, OpenMP*, SYCL*)



<https://github.com/oneapi-src/oneDPL>

Parallel Algorithms API

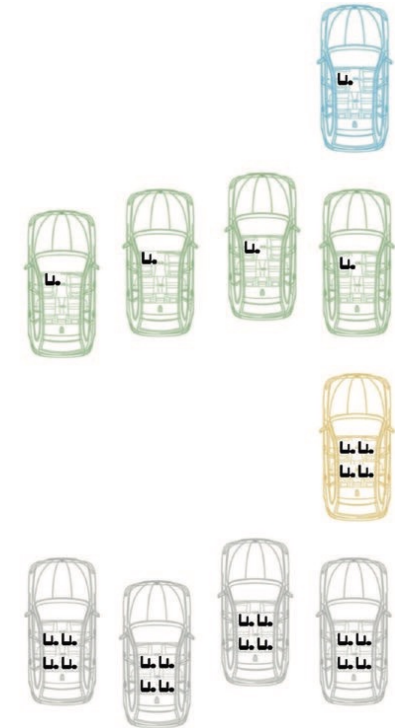
An overview

Parallel Algorithms API: Brief Overview

- C++17/20 parallel algorithms
 - Execution policies define how/where to run standard algorithms
 - CPU execution: oneTBB and OpenMP*
 - GPU & accelerators: SYCL back-end support for all algorithms with execution policies that are part of C++17
- Open-source upstream (LLVM*) implementation named Parallel STL is derived from Intel's implementation for CPU

STL Algorithms with Execution Policies

- **standard sequential sort**
`sort(v.begin(), v.end());`
- **explicitly sequential sort**
`sort(execution::seq, v.begin(), v.end());`
- **permitting parallel execution**
`sort(execution::par, v.begin(), v.end());`
- **permitting vectorization only (no parallel execution)**
`sort(execution::unseq, v.begin(), v.end());`
- **permitting parallel execution and vectorization**
`sort(execution::par_unseq, v.begin(), v.end());`



multilane highway analogy from
ProTBB book, M. Voss et al.

Parallel STL in C++

```
/* ... a for loop */  
for(auto &i : v) {  
    i++;  
}
```

```
#include <algorithm>  
/* ... for each algorithm w/o execution policy */  
std::for_each(v.begin(), v.end(), [](float &i){i++;});
```

```
#include <algorithm>  
#include <execution>  
/* ... for each algorithm with parallel execution policy */  
std::for_each(std::execution::par, v.begin(), v.end(), [](float &i){i++;});
```

SYCL support and utilities

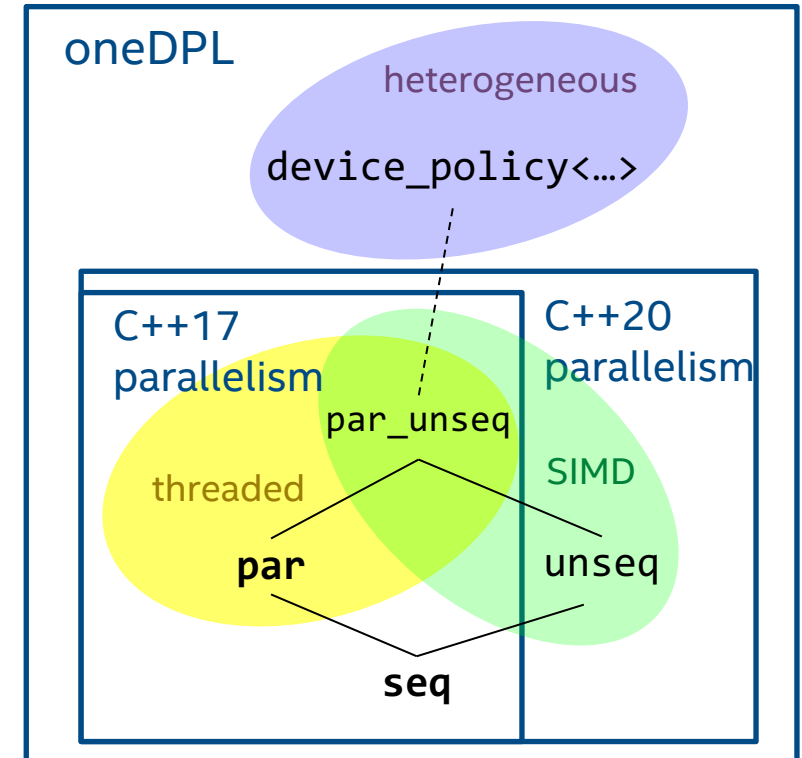
An overview

DPC++ Execution Policies



Spec

- Specifies where and how an algorithm runs
 - Encapsulates a SYCL queue
 - The queue defines where the Parallel algorithm executes
- Policy usage:
 - Include `<oneapi/dpl/execution>` in your code
 - Create a policy object by providing a policy type, a (optional) class type for unique kernel name, and one of the following constructor arguments:
 - A SYCL queue
 - A SYCL device
 - An existing policy object
 - Pass the created policy object to a Parallel algorithm
- **`oneapi::dpl::execution::dpcpp_default`**
 - Predefined object of the `device_policy` class with default kernel name and default queue
 - Can be directly passed when invoking an algorithm



How to Target a Device:



Source
Code

First simple example (to be continued ...)

```
#include <oneapi/dpl/execution>
#include <oneapi/dpl/algorithm>

int main()
{
    using namespace oneapi;
    std::vector<int> vec(1000);
    dpl::fill(dpl::execution::dpcpp_default,
              vec.begin(), vec.end(), 42);
    return 0;
}

// To build: icpx -fsycl <example_name>.cpp -o test
```



Spec

Buffer Wrapper

`oneapi::dpl::begin` and `oneapi::dpl::end` are helper functions for passing DPC++ buffers to oneDPL algorithms

These functions accept a buffer and return an object of an unspecified type that satisfies the following requirements:

- It is CopyConstructible, CopyAssignable, and comparable with operators `==` and `!=`;
- The following expressions are valid: `a + n`, `a - n`, `a - b`, where `a` and `b` are objects of the type and `n` is an integer value
- It provides the `get_buffer()` method that returns the buffer passed to the `begin` or `end` function

Simple Example using SYCL Buffer



Source
Code

Keeping data on the device with consecutive usage

```
#include <oneapi/dpl/execution>
#include <oneapi/dpl/algorithm>
#include <oneapi/dpl/iterator>
#include <CL/sycl.hpp>
int main()
{
    sycl::buffer<int> buf{ 1000 };
    dpl::fill(dpl::execution::dpcpp_default, dpl::begin(buf),
              dpl::end(buf), 42);
    auto result = dpl::find(dpl::execution::dpcpp_default,
                           dpl::begin(buf), dpl::end(buf), 42);
    return 0;
}
```

Example with SYCL Unified Shared Memory (USM)

Directly pass USM pointers to parallel algorithms

```
#include <oneapi/dpl/execution>
#include <oneapi/dpl/algorithm>
#include <CL/sycl.hpp>
int main()
{
    sycl::queue q;
    const int n = 1000;
    int* data = sycl::malloc_shared<int>(n, q);
    auto my_policy = dpl::execution::make_device_policy<class Fill>(q);
    dpl::fill(my_policy, data, data + n, 42);
    // q.wait();
    sycl::free(data, q);

    return 0;
}
```

C++ for Device Programming

Using the C++ Standard Library in SYCL*

■ Host

- Code running on the host CPU can use everything

■ Device

- Some std classes/functions cannot work in device kernel code due to SYCL* restrictions
 - Items using exception
 - Dynamic memory allocation
 - Virtual functions

Tested Standard C++ APIs

- ~200 standard C++ APIs have been tested
 - Usage in SYCL* kernels
 - Data transfer between host and device (only specific C++ types)
- Across 3 major implementations:
 - libstdc++(GNU): deployed in most Linux distributions
 - libc++(LLVM): macOS and FreeBSD
 - Microsoft STL: Windows; shipped with Microsoft Visual Studio
- Available in both `namespace std` and `namespace oneapi::dpl`

Visit [Intel® oneAPI DPC++ Library Guide](#) for a detailed list

Example: Complex Numbers

use of std components on the device

```
#include <oneapi/dpl/execution>
#include <oneapi/dpl/algorithm>
#include <oneapi/dpl/iterator>
#include <CL/sycl.hpp>

#include <complex>

int main() {
    const size_t n = 100;
    sycl::buffer<std::complex<double>> c_buf{n};

    auto my_policy = dpl::execution::dpcpp_default;
    auto index = dpl::counting_iterator<int>(0);

    dpl::transform(my_policy, index, index + n, dpl::begin(c_buf), [=](const auto& i) {
        const double v = fabs(static_cast<double>(n)/2.0 - static_cast<double>(i));
        return std::complex<double>{v, v};
    });

    return 0;
}
```

Custom Iterators

Expanding Applicability of Parallel Algorithms with Custom Iterators

Custom Iterators:

- `counting_iterator`
- `zip_iterator`
- `transform_iterator`
- `permutation_iterator`
- `discard_iterator`

Counting Iterator

`0` `1` `2` `3` $\longrightarrow$ Integer sequence

- Represents a linear, increasing sequence of integer values
- Commonly used as an index:
`[](auto i){a[i] = b[i];}`
- Advantage: Not in memory



Spec

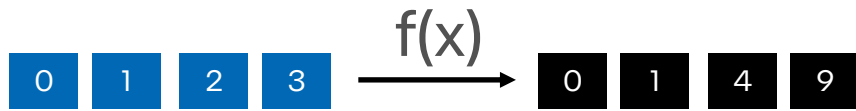
*Custom iterators inspired by Boost.Compute**

Increase Adaptability with Custom Iterators



Spec

Transform Iterator



- Applies transformation to dereferenced values of another iterator

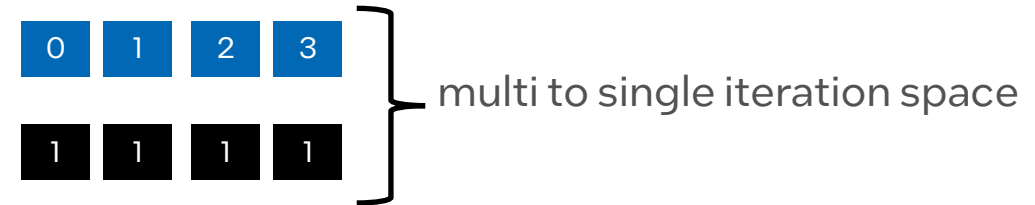
- Example: Square numbers
- Input iterator

Here: counting iterator

- Unary function

Here: `[](auto& x){return x*x;}`

Zip Iterator



- Combines multiple iterators into a single iteration space
 - Extends applicability of STL algorithms
 - Combinations of custom iterators are possible (permutation, counting, ...)
 - Make function accepts an argument pack

*Custom iterators inspired by Boost.Compute**

How to Use a Zip Iterator

$$A += B + s * C$$

TriAdd example

```
/* ... */
double *d_A = sycl::malloc_device<double>(length, q);
double *d_B = sycl::malloc_device<double>(length, q);
double *d_C = sycl::malloc_device<double>(length, q);
/* ... */
double scalar(3);
{
    for (int iter = 0; iter <= iterations; iter++) {
        if (iter == 1)
            auto begin = dpl::make_zip_iterator(d_A, d_B, d_C);
            dpl::transform(dpl::execution::make_device_policy<class TriAdd>(q), begin,
                           begin + length, d_A, [=](const auto &t) {
                               using std::get;
                               return get<0>(t) + get<1>(t) + scalar * get<2>(t);
                           });
    }
}
/* ... */
```

<https://github.com/ParRes/Kernels/blob/default/Cxx11/nstream-onedpl.cc>

Zip Iterator with structured binding (C++17)

TriAdd example

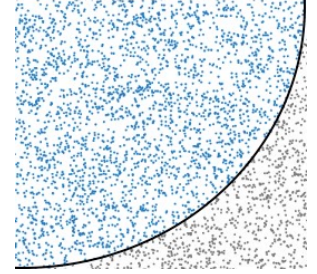
```
/* ... */
double *d_A = sycl::malloc_device<double>(length, q);
double *d_B = sycl::malloc_device<double>(length, q);
double *d_C = sycl::malloc_device<double>(length, q);
/* ... */
double scalar(3);
{
    for (int iter = 0; iter <= iterations; iter++) {
        if (iter == 1)
            auto begin = dpl::make_zip_iterator(d_A, d_B, d_C);
            dpl::transform(dpl::execution::make_device_policy<class TriAdd>(q), begin,
                           begin + length, d_A, [=](const auto &t) {
                               auto [a, b, c] = t;
                               return a + b + scalar * c;
                           });
    }
}
/* ... */
```

https://en.cppreference.com/w/cpp/language/structured_binding

Random sampling example: Monte Carlo Pi

Making full use of oneDPL

```
int main() {
    sycl::queue q; //(sycl::gpu_selector{});
    auto my_policy = dpl::execution::make_device_policy(q);
    auto sum = dpl::transform_reduce( my_policy, dpl::counting_iterator<int>(0),
                                     dpl::counting_iterator<int>(N), 0, std::plus<float>{},
                                     [=](auto n){
    float local_sum = 0.0f;
    // Get random coords
    dpl::minstd_rand engine(SEED, 2*n*LOCAL_N);
    dpl::uniform_real_distribution<float> distr(-1.0f,1.0f);
    for(int i = 0; i < LOCAL_N; ++i) {
        float x = distr(engine);
        float y = distr(engine);
        auto hypotenuse_sqr = x * x + y * y;
        if (hypotenuse_sqr <= 1.0)
            local_sum += 1.0;
    }
    return local_sum / (float)LOCAL_N;
    }));
    estimated_pi = 4.0 * (float)sum / N;
    return 0;
}
```



Tips and Tricks for effective use

Optimization Strategies

- Optimize common code path
- 80/20 rule – e.g., 80% of the time is spend in 20% of the code
- Spend your time optimizing the important part
 - Even if that means you slow down code rarely executed
- Best and most important way of optimization is picking a good algorithm
- $O(n^2)$ vs. $O(n \cdot \log)$
- small problem sizes → linear time could beat cubic time

General Recommendations

- Parallel execution profitable if overhead $< 50\%$
 - Refers to Amdahl's Law https://en.wikipedia.org/wiki/Amdahl%27s_law
- Take memory access of algorithm into account
 - Caches and memory copy (implicit and explicit)
- Vectorize at innermost level
- Parallelize at outermost level
- Consider parallel execution at levels in between if overhead is acceptable
 - Not applicable for device execution (SYCL restrictions apply)

Summary

- Productivity library for heterogeneous computing
 - Expressing higher-level parallel pattern with Parallel API
 - Composes well with SYCL
 - Target compute devices with custom policies

References

oneDPL specification

<https://spec.oneapi.io/versions/latest/elements/oneDPL/source/index.html>

oneDPL Library Guide

<https://docs.oneapi.io/versions/latest/onedpl/index.html>

oneDPL source code

<https://github.com/oneapi-src/oneDPL>

Build & optimize your code in the Intel® DevCloud

Intel® oneAPI preinstalled and ready to go: <https://devcloud.intel.com/oneapi/>

Notices & Disclaimers

- Intel technologies may require enabled hardware, software or service activation
- Performance varies by use, configuration and other factors. Learn more on the Performance Index site.
- No product or component can be absolutely secure.
- All product plans and roadmaps are subject to change without notice
- © Intel Corporation. Intel, the Intel logo, and other Intel marks are trademarks of Intel Corporation or its subsidiaries. Other names and brands may be claimed as the property of others. SYCL is a registered trademark of the Khronos Group, Inc.

